

getElementsByTagName()

`document.getElementsByTagName()` is a JavaScript method used to **select HTML elements by their tag name**.

It returns a **collection (like a list)** of all elements with that tag name.

Simple Syntax

```
document.getElementsByTagName("tagname");
```

"tagname" → The name of the HTML tag (like "p", "div", "h1", "li", etc.)

Important Things to Remember

1. It returns **multiple elements** (even if there is only one).
2. The return type is an **HTMLCollection** (similar to array).
3. You must use **index number** to access elements.
4. Index starts from **0**.

Example 1: Selecting <p> tags

```
<!DOCTYPE html>
<html>
<head>
  <title>Example 1</title>
</head>
<body>

<h2>Example 1: Change First Paragraph</h2>

<p>First paragraph</p>
<p>Second paragraph</p>
<p>Third paragraph</p>

<button onclick="changeFirst()">Change First Paragraph</button>

<script>
function changeFirst() {
  let paragraphs = document.getElementsByTagName("p");
  paragraphs[0].innerHTML = "First paragraph changed!";
}
</script>
```

```
</body>
</html>
```

Here:

- paragraphs[0] → first <p>
- paragraphs[1] → second <p>

Example 2: Changing Text of All <p> Tags

```
<!DOCTYPE html>
<html>
<head>
  <title>Example 2</title>
</head>
<body>

<h2>Example 2: Change All Paragraphs</h2>
<p>Paragraph 1</p>
<p>Paragraph 2</p>
<p>Paragraph 3</p>

<button onclick="changeAll()">Change All Paragraphs</button>
<script>
function changeAll() {
  let paragraphs = document.getElementsByTagName("p");
  for (let i = 0; i < paragraphs.length; i++) {
    paragraphs[i].innerHTML = "All paragraphs changed!";
  }
}
</script>
</body>
</html>
```

Example 3: Changing Background Color of All <div>

```
<!DOCTYPE html>
<html>
<head>
  <title>Example 3</title>
</head>
<body>

<h2>Example 3: Change Div Background</h2>

<div style="width:100px;height:50px;border:1px solid black;">Box 1</div>
<div style="width:100px;height:50px;border:1px solid black;">Box 2</div>
<div style="width:100px;height:50px;border:1px solid black;">Box 3</div>

<br>
<button onclick="changeColor()">Change Background</button>

<script>
function changeColor() {
  let divs = document.getElementsByTagName("div");

  for (let i = 0; i < divs.length; i++) {
    divs[i].style.backgroundColor = "yellow";
  }
}
</script>

</body>
</html>
```

Example 4: If Only One Element Exists

Even if there is only one <h1>:

```
<h1>Hello World</h1>
```

You still need index:

```
let heading = document.getElementsByTagName("h1");
```

```
heading[0].style.color = "red";
```

Wrong:

```
heading.style.color = "red"; // Error
```

```
<!DOCTYPE html>
<html>
<head>
  <title>Example 4</title>
</head>
<body>

<h1>This is a Heading</h1>

<button onclick="changeHeading()">Change Heading Color</button>

<script>
function changeHeading() {
  let heading = document.getElementsByTagName("h1");
  heading[0].style.color = "red";
}
</script>

</body>
</html>
```

Special Case: Select ALL Elements

let allElements = document.getElementsByTagName("*");
document.getElementsByTagName("*") selects all elements present in the HTML document.

"*" selects **all elements in the page**.

```
<!DOCTYPE html>
<html>
<head>
  <title>Select All Elements Example</title>
</head>
<body>

<h1>Main Heading</h1>

<p>This is paragraph 1</p>
```

```

<p>This is paragraph 2</p>

<div style="width:150px;height:50px;border:1px solid black;">
  This is a div
</div>

<span>This is a span text</span>

<br><br>

<button onclick="changeAllElements()">Change All Elements</button>

<script>
function changeAllElements() {

  // Select ALL elements in the page
  let allElements = document.getElementsByTagName("*");

  // Loop through all elements
  for (let i = 0; i < allElements.length; i++) {
    allElements[i].style.backgroundColor = "lightblue";
    allElements[i].style.color = "red";
  }
}
</script>

</body>
</html>

```

What is document.querySelector()?

It is a JavaScript method used to **select the first matching HTML element**.

It can select using:

- **id**
- **class**
- **tag name**
- **CSS selector**

It always selects **ONLY THE FIRST MATCHING ELEMENT**.

Simple Syntax

```
document.querySelector("selector");
```

The selector works like CSS.

Example 1: Select by ID

HTML

```
<p id="demo">Hello World</p>
```

JavaScript

```
let element = document.querySelector("#demo");  
element.style.color = "red";
```

means ID.

Example 2: Select by Class

```
<p class="box">Paragraph 1</p>  
<p class="box">Paragraph 2</p>  
let element = document.querySelector(".box");  
element.style.color = "blue";
```

Only the **first** .box will change.

. means class.

Example 3: Select by Tag

```
let element = document.querySelector("p");  
it selects the first paragraph only.
```

Difference from `getElementsByTagName()`

<code>querySelector()</code>	<code>getElementsByTagName()</code>
Selects first match	Selects all elements
Uses CSS selector	Uses tag name only
No index needed	Index needed

`document.querySelector()` selects the first HTML element that matches a specified CSS selector.

Program: Change First Matching Element

```
<!DOCTYPE html>
<html>
<head>
  <title>querySelector Example</title>
  <style>
    .box {
      border: 1px solid black;
      padding: 10px;
      margin: 5px;
    }
  </style>
</head>
<body>

<h2 id="heading">This is a Heading</h2>

<p class="box">Paragraph 1</p>
<p class="box">Paragraph 2</p>
<p class="box">Paragraph 3</p>

<button onclick="changeElement()">Click Me</button>

<script>
function changeElement() {

  // Select first element with class "box"
  let element = document.querySelector(".box");

  element.style.backgroundColor = "yellow";
  element.style.color = "red";
  element.innerHTML = "Changed using querySelector!";
}
</script>

</body>
</html>
```

`document.querySelectorAll()`

What is `document.querySelectorAll()`?

It is a JavaScript method used to **select ALL elements that match a CSS selector**.

It works like `querySelector()`

But the difference is:

- `querySelector()` → selects **first match only**
- `querySelectorAll()` → selects **ALL matching elements**

Syntax

```
document.querySelectorAll("selector");
```

It returns a **NodeList (like an array)**.

So we usually use a **loop** to access elements.

Example 1: Select All Paragraphs

HTML

```
<p>Paragraph 1</p>
<p>Paragraph 2</p>
<p>Paragraph 3</p>
```

JavaScript

```
let paragraphs = document.querySelectorAll("p");
```

```
for (let i = 0; i < paragraphs.length; i++) {
  paragraphs[i].style.color = "red";
}
```

All paragraphs turn red.

Example 2: Select All Elements by Class

```
<p class="box">Box 1</p>
<p class="box">Box 2</p>
<p class="box">Box 3</p>
```



```
let boxes = document.querySelectorAll(".box");
```

. means class

All .box elements will be selected.

Example 3: Select by ID

Even though ID should be unique:

```
document.querySelector("#demo");
```

It still returns a NodeList (usually with one element).

Difference Table

<code>querySelector()</code>	<code>querySelectorAll()</code>
First match only	All matches
Returns single element	Returns NodeList
No loop needed	Loop needed

`document.querySelectorAll()` selects all HTML elements that match a specified CSS selector and returns them as a NodeList.

Program: Change All Matching Elements

```
<!DOCTYPE html>
<html>
<head>
  <title>querySelectorAll Example</title>
  <style>
    .box {
      border: 1px solid black;
      padding: 10px;
      margin: 5px;
    }
  </style>
</head>
<body>
```

```

<h2>querySelectorAll() Demo</h2>

<p class="box">Paragraph 1</p>
<p class="box">Paragraph 2</p>
<p class="box">Paragraph 3</p>

<button onclick="changeAll()">Click Me</button>

<script>
function changeAll() {

    // Select ALL elements with class "box"
    let elements = document.querySelectorAll(".box");

    // Loop through them
    for (let i = 0; i < elements.length; i++) {
        elements[i].style.backgroundColor = "yellow";
        elements[i].style.color = "blue";
        elements[i].innerHTML = "Changed using querySelectorAll!";
    }
}
</script>

</body>
</html>

```

querySelectorAll() with ID

Even though **ID should be unique**,
 querySelectorAll("#demo") still returns a **NodeList**.

Usually → it contains **only one element**.

```

<!DOCTYPE html>
<html>
<head>
  <title>querySelectorAll with ID</title>
  <style>
    #demo {
      padding: 10px;
      border: 1px solid black;
    }
  </style>

```

```

        width: 200px;
    }
</style>
</head>
<body>

<h2>querySelectorAll() with ID Example</h2>

<p id="demo">This is a paragraph with ID demo</p>

<button onclick="changeElement()">Click Me</button>

<script>
function changeElement() {

    // Select element using ID with querySelectorAll
    let element = document.querySelectorAll("#demo");

    // It returns a NodeList (like array)
    console.log(element); // You can check in browser console

    // Access first element using index
    element[0].style.backgroundColor = "yellow";
    element[0].style.color = "red";
    element[0].innerHTML = "Changed using querySelectorAll() with ID!";
}
</script>

</body>
</html>

```

querySelectorAll() always returns a NodeList, even if only one element matches. NodeList is like an array that stores multiple HTML elements.

What is a “Node”?

In HTML DOM:

- Every element is a node
- <p> is a node
- <div> is a node
- <h1> is a node
- Even text inside tags is a node

Changing Content

1. innerHTML

Changes HTML content.

Changes HTML content (it understands HTML tags)

```
document.getElementById("demo").innerHTML = "<b>Hello</b>";
```

```
<!DOCTYPE html>
<html>
<head>
  <title>innerHTML Example</title>
</head>
<body>

<h2>innerHTML Demo</h2>

<p id="demo">Original Text</p>

<button onclick="changeHTML()">Click Me</button>

<script>
function changeHTML() {
  document.getElementById("demo").innerHTML = "<b>Hello</b> <i>World</i>";
}
</script>

</body>
</html>
```

When button is clicked:

Text becomes **bold** and *italic*

Because innerHTML understands HTML tags

2. innerText

Changes only text.

```
document.getElementById("demo").innerText = "Hello World";
```

```
<!DOCTYPE html>
<html>
<head>
  <title>innerText Example</title>
</head>
<body>

<h2>innerText Demo</h2>

<p id="demo">Original Text</p>

<button onclick="changeText()">Click Me</button>

<script>
function changeText() {
  document.getElementById("demo").innerText = "<b>Hello World</b>";
}
</script>

</body>
</html>
```

What Happens?

It will display:

Hello World

Not bold

Because innerText treats everything as plain text.

3. textContent

Also changes text (more consistent than innerText).

```
document.getElementById("demo").textContent = "New Text";
```

```
<!DOCTYPE html>
<html>
<head>
  <title>textContent Example</title>
</head>
```

```
<body>

<h2>textContent Demo</h2>

<p id="demo">Original Text</p>

<button onclick="changeContent()">Click Me</button>

<script>
function changeContent() {
    document.getElementById("demo").textContent = "<b>New Text</b>";
}
</script>

</body>
</html>
```

It will display:

New Text

Not bold

Because textContent does not parse HTML.

- **innerHTML** → Changes HTML content including tags
- **innerText** → Changes only visible text
- **textContent** → Changes text content (more consistent)

Property	Understands HTML?	Use Case
innerHTML	Yes	Add HTML tags dynamically
innerText	No	Change visible text only
textContent	No	Change text (recommended)

Changing Attributes

1. `setAttribute()`

Used to **add or change an attribute** of an element.

```
document.getElementById("img").setAttribute("width", "300");
<!DOCTYPE html>
<html>
<head>
  <title>setAttribute Example</title>
</head>
<body>

<h2>setAttribute() Demo</h2>



<br><br>

<button onclick="changeWidth()">Change Width</button>

<script>
function changeWidth() {
  document.getElementById("img").setAttribute("width", "300");
}
</script>

</body>
</html>
```

2. `getAttribute()`

Used to **get (read) the value of an attribute**.

```
let value = document.getElementById("img").getAttribute("src");
<!DOCTYPE html>
<html>
<head>
  <title>getAttribute Example</title>
</head>
<body>

<h2>getAttribute() Demo</h2>


```

```

<br><br>

<button onclick="showSource()">Show Image Source</button>

<p id="result"></p>

<script>
function showSource() {
    let value = document.getElementById("img").getAttribute("src");
    document.getElementById("result").innerText = "Image Source: " + value;
}
</script>

</body>
</html>

```

3. removeAttribute()

Used to **remove an attribute** from an element.

```

document.getElementById("img").removeAttribute("width");
<!DOCTYPE html>
<html>
<head>
    <title>removeAttribute Example</title>
</head>
<body>

<h2>removeAttribute() Demo</h2>



<br><br>

<button onclick="removeWidth()">Remove Width</button>

<script>
function removeWidth() {
    document.getElementById("img").removeAttribute("width");
}
</script>
</body>
</html>

```


Changing CSS Style

1. Direct style

```
document.getElementById("box").style.backgroundColor = "red";  
document.getElementById("box").style.width = "200px";
```

Must use **camelCase**:

- backgroundColor
- fontSize
- marginTop

classList

Add or remove CSS classes.

classList is used to:

- Add a class
- Remove a class
- Toggle (add if not present, remove if present)

```
let box = document.getElementById("box");
```

```
box.classList.add("active");
```

```
box.classList.remove("active");
```

```
box.classList.toggle("active");
```

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>classList Example</title>  
  
  <style>  
    #box {  
      width: 200px;  
      height: 100px;  
      background-color: lightgray;
```

```
        text-align: center;
        line-height: 100px;
        margin-bottom: 15px;
        border: 2px solid black;
    }

    .active {
        background-color: green;
        color: white;
        font-weight: bold;
        border-radius: 10px;
    }
</style>
</head>
<body>

<h2>classList Demo</h2>

<div id="box">Box</div>

<button onclick="addClass()">Add Class</button>
<button onclick="removeClass()">Remove Class</button>
<button onclick="toggleClass()">Toggle Class</button>

<script>
let box = document.getElementById("box");

function addClass() {
    box.classList.add("active");
}

function removeClass() {
    box.classList.remove("active");
}

function toggleClass() {
    box.classList.toggle("active");
}
```

```
</script>
```

```
</body>
```

```
</html>
```

`classList` is used to add, remove, or toggle CSS classes of an element dynamically.